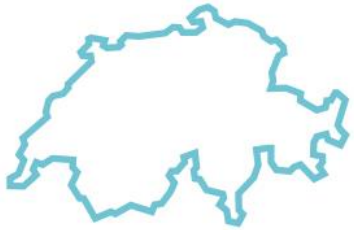
die Mobiliar

Umgang mit Legacy in der digitalen Transformation

BATBern 45, Andreas Grütter, Leiter IT Architektur

Die Mobiliar im Überblick

die Mobiliar



Nr. 1 in der Schweiz für Haushalt-, Betriebs- und Risikolebensversicherungen

Gründungsjahr 1826

rund **5400** Mitarbeitende

davon **300** Lernende

160 Standorte

Die 79 Generalagenturen sind mit ihren rund 80 Agenturen immer nahe bei unseren Kunden.

9 von 10 Schadenfällen

erledigen unsere Generalagenturen vor Ort.

CHF 175 Mio.

Diesen Betrag schütten wir als freiwillige Überschussbeteiligung an unsere Kunden aus.

24/7

Die Mobiliar 24 h Assistance steht unseren Kunden rund um die Uhr an 365 Tagen im Jahr zur Verfügung.

rund **CHF 4 Mia.**

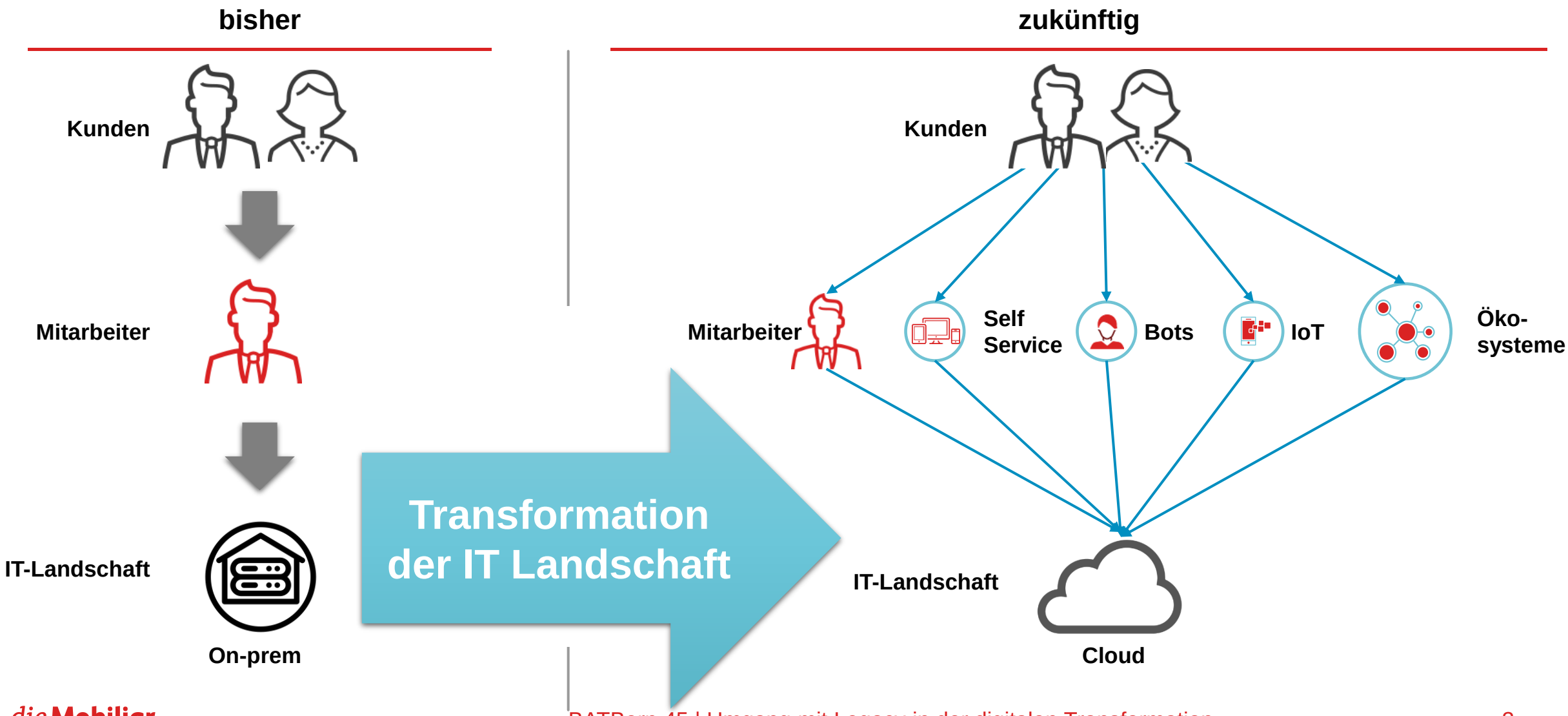
Im Berichtsjahr konnten wir das Prämienvolumen um 3.7% steigern

CHF 488.6 Mio.

Die Mobiliar weist erneut einen ausgezeichneten Gewinn aus.

2 Mio. Privatpersonen und Unternehmen schenken uns ihr Vertrauen.

Digitalisierung erfordert eine Transformation unserer IT-Landschaft



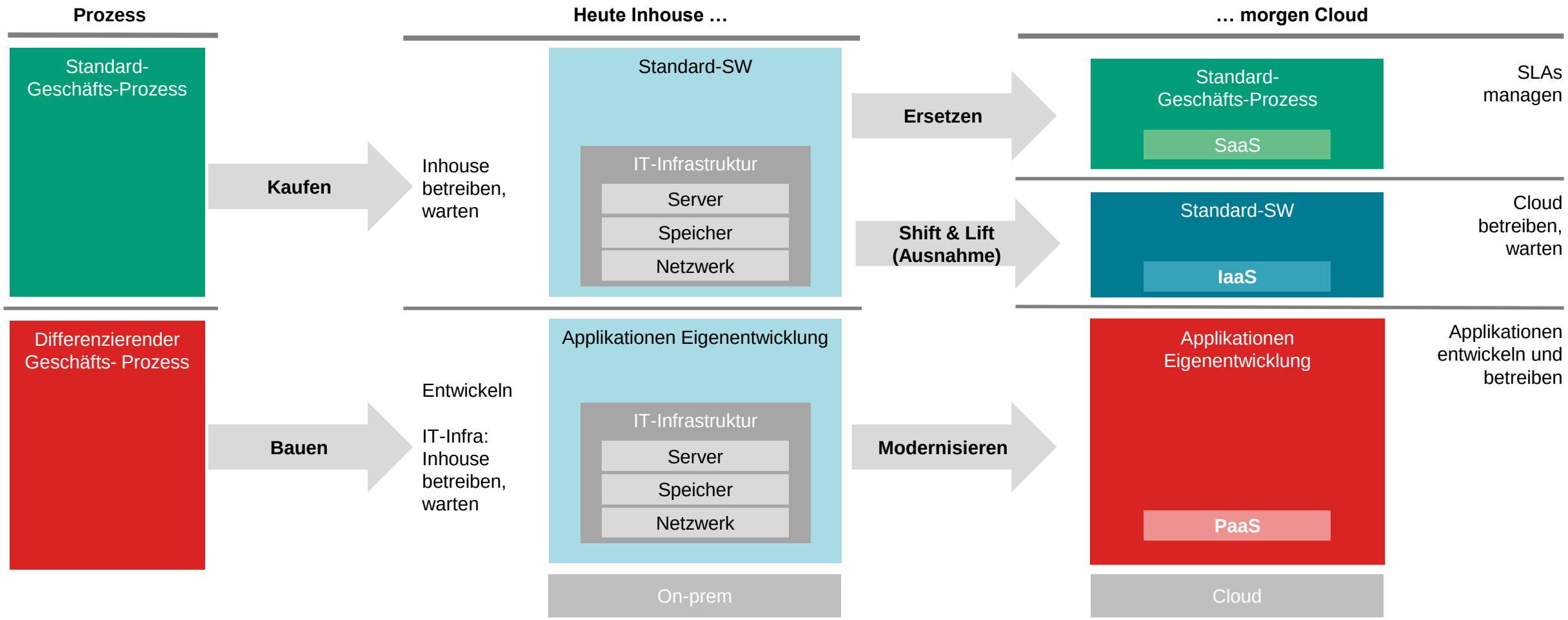
Digitale Transformation der Mobiliar…

Konsistentes Kundenerlebnis über alle on- und offline Kanäle

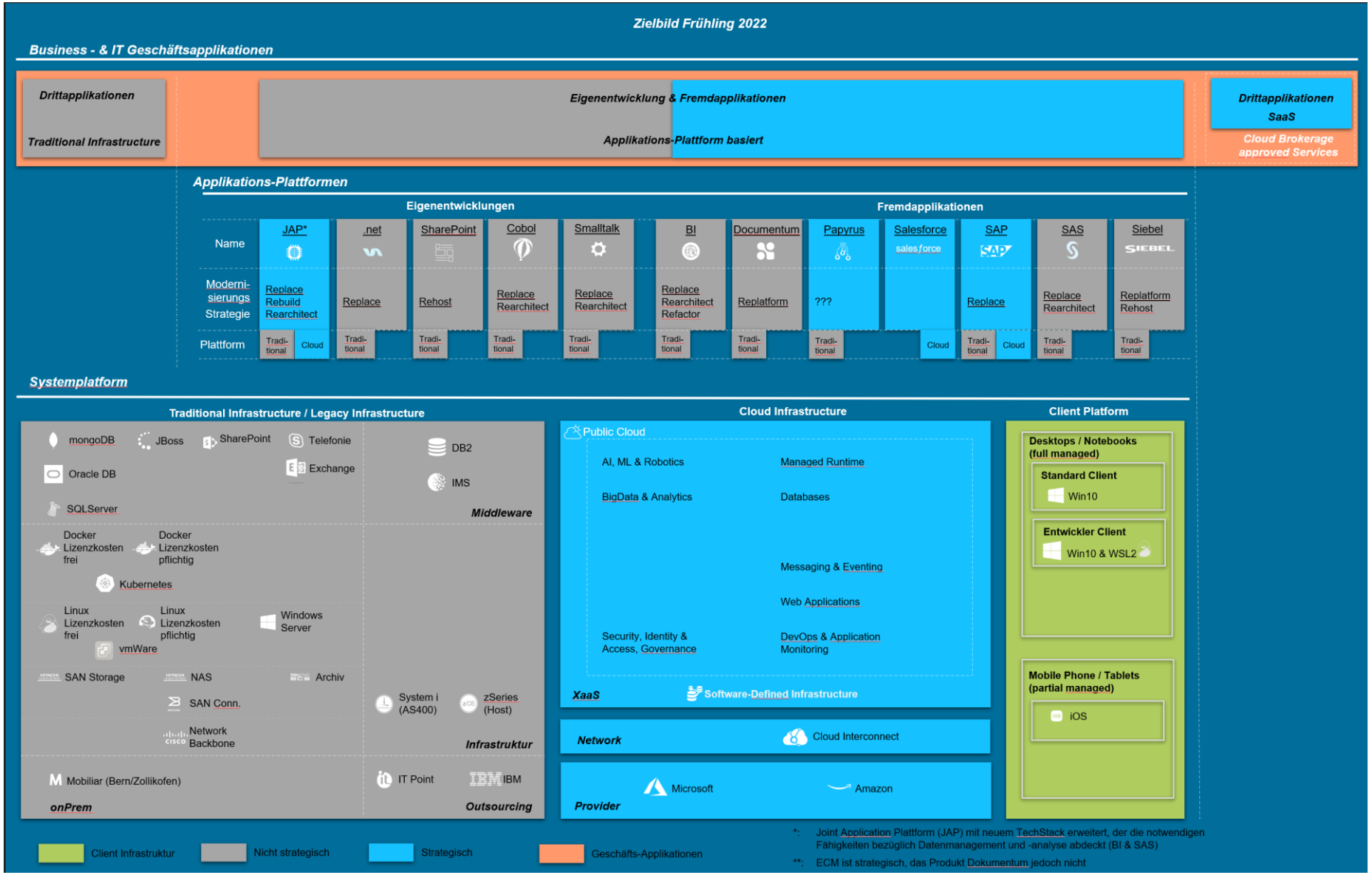
Das bedeutet erneuerte…

- Kanäle
- Produkte
- Prozesse
- Technologien
- Fähigkeiten / Vorgehen / Praktiken

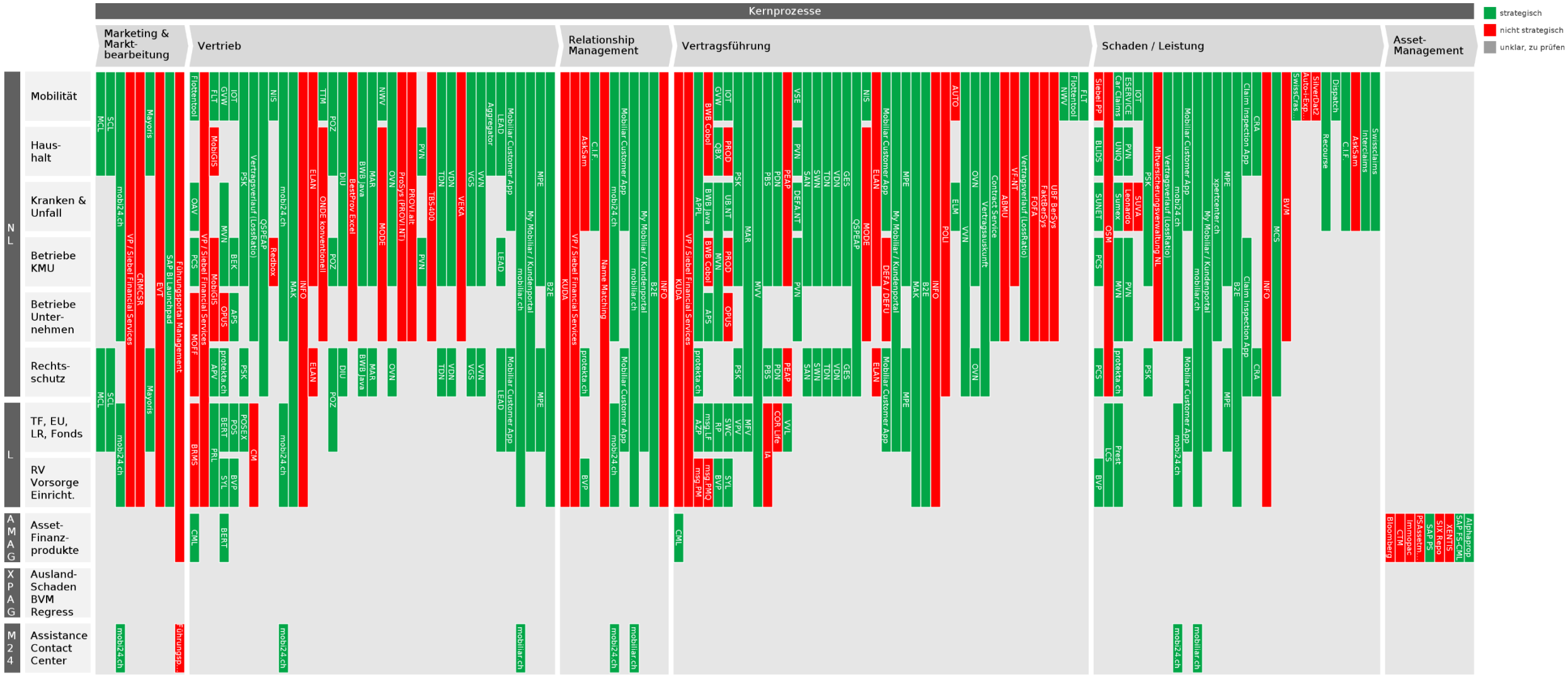
Bezüglich Technologie haben wir eine klare Cloud Strategie



Aktuelle Situation unserer Technologie Landschaft

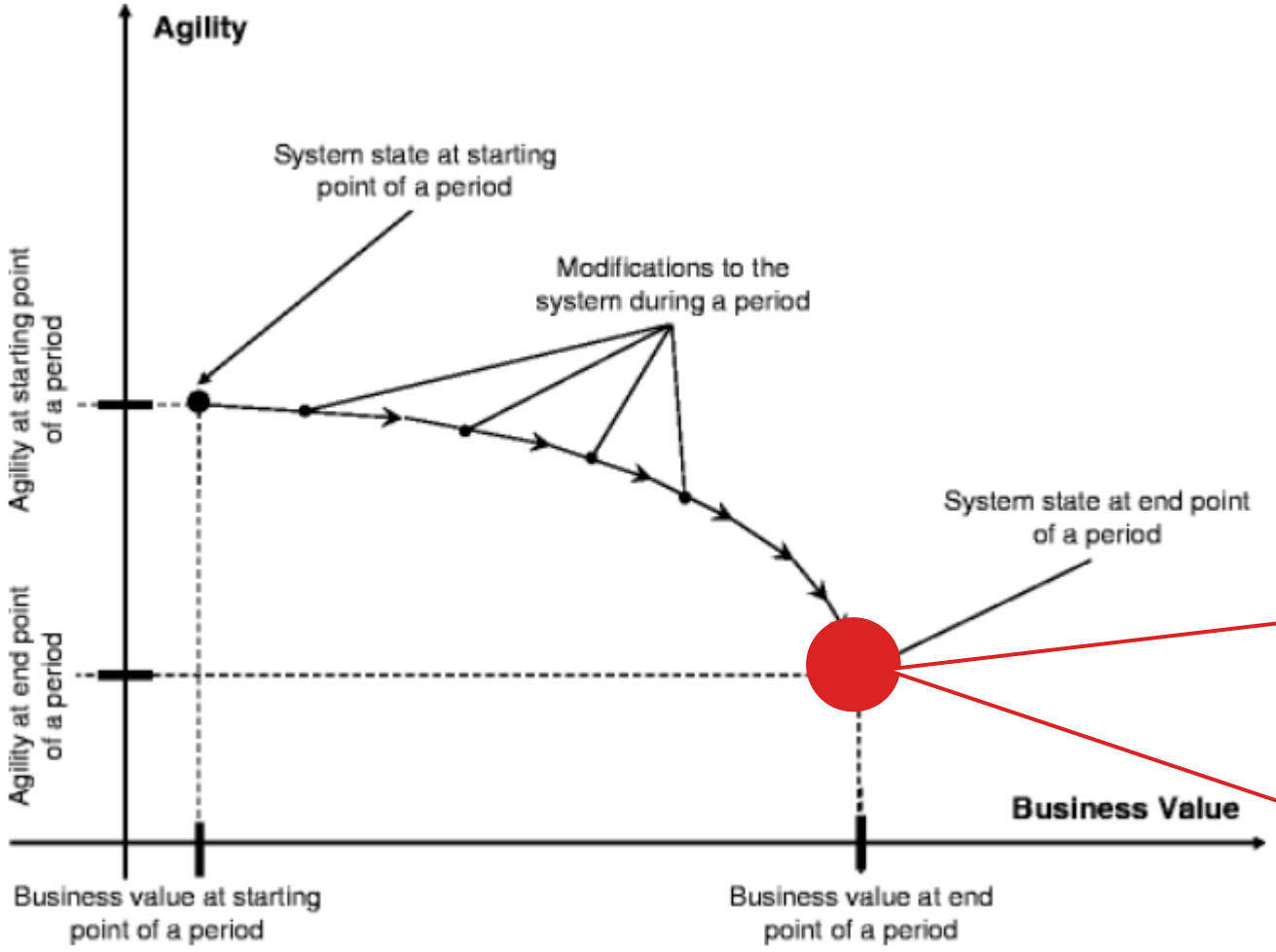


Zahlreiche Mission Critical Systeme basieren auf Legacy Technologien



BUSA Prozessmodell: https://cwiki.mobicorp.ch/confluence/display/BUSA/Prozessmodell+-+Logik+des+Aufbaus

Legacy System: Etablierte, historisch gewachsene Anwendung

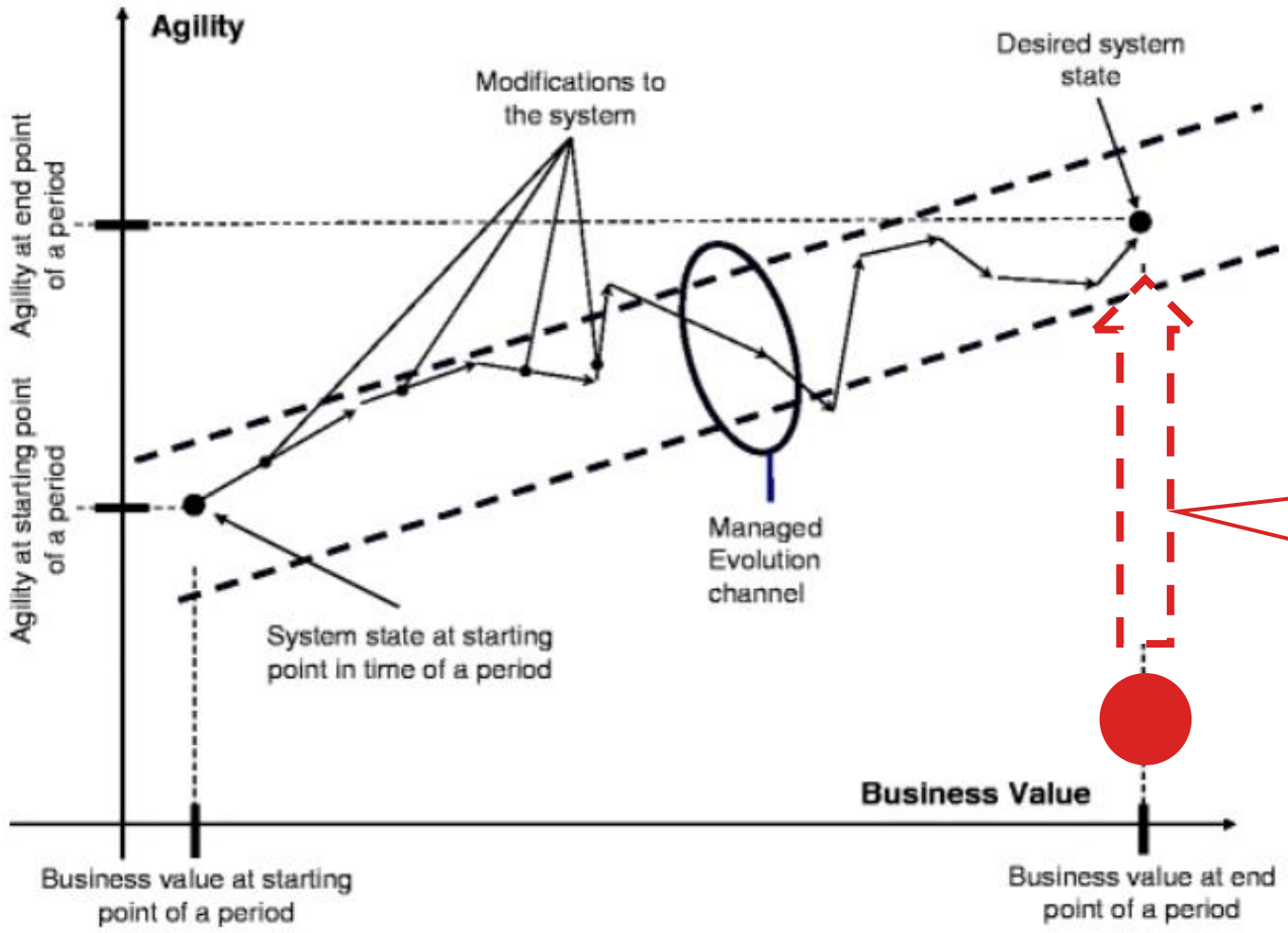


Wichtigste Ursachen für den Agilitätsverslust:

- **Historisch gewachsene Komplexität**
- **Knowhow Verlust z.B. wegen unzureichender Doku**
- **Abnehmende Ressourcen Verfügbarkeit**
- **Veraltete Technologie**

Quelle: Managed Evolution

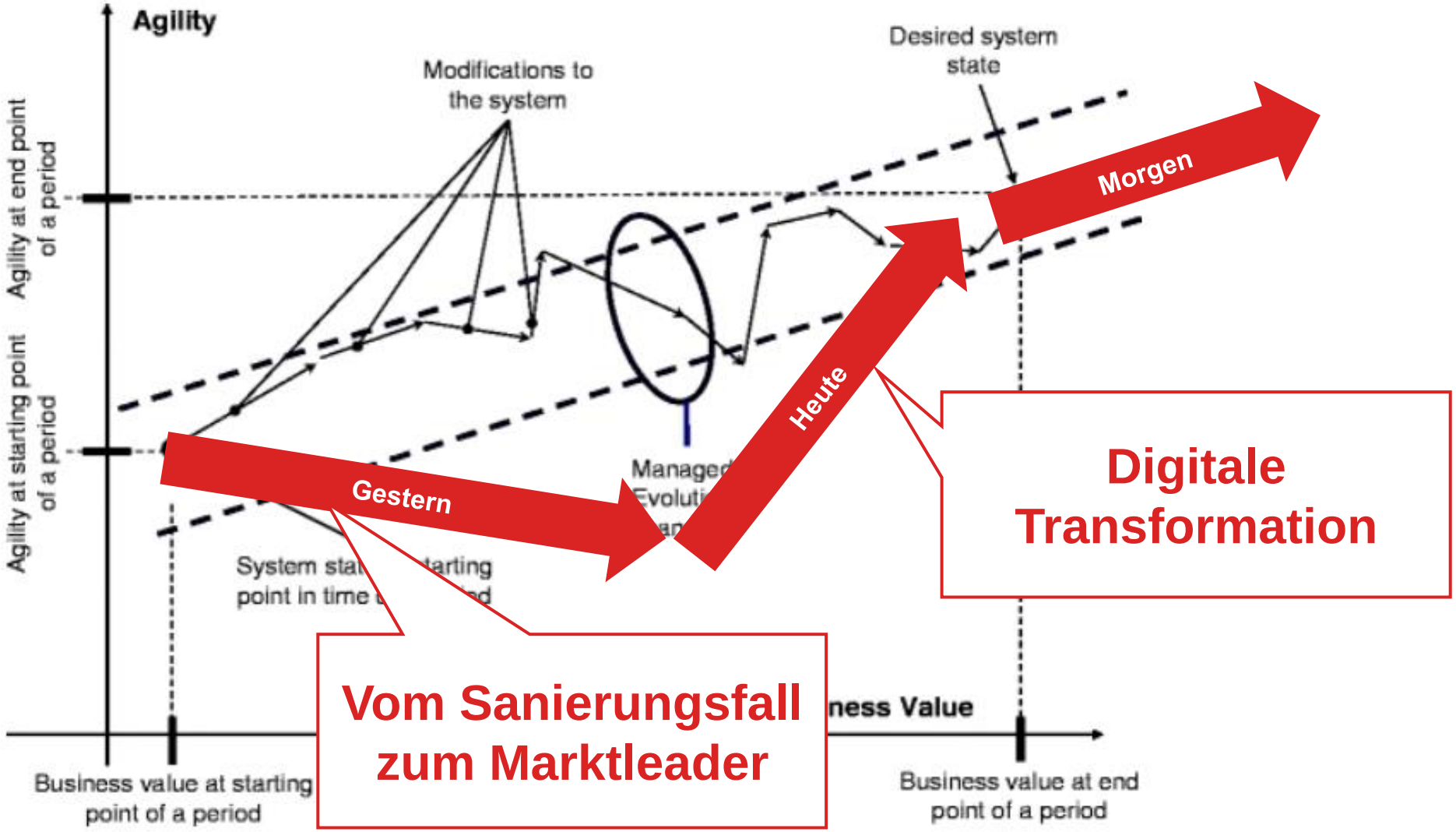
Problem von Legacy Systemen



Massives Investment ohne added Business Value

Quelle: Managed Evolution

Von der Mobi Evolution zu Managed Evolution



Quelle: Managed Evolution

Fachlich getriebene Modernisierung entlang der Produktlinien (vereinfacht)

Die wichtigsten Architektur Prinzipien zur Beherrschung der Komplexität

Zukunfts-orientierung

Keine Kompromisse in neuen Systemen aufgrund der Legacy Systeme

Redundanzfreiheit

Daten haben einen einzigen Master

Konsequenz des Produktweisen Vorgehens für die Altsysteme

Abschaltung der Altsysteme ist erst möglich, wenn alle Produktlinien modernisiert sind.

Vor- und Nachteile des Produktweisen Vorgehen

Vorteile

- **Fokus auf Business Innovationen**
- **Neue innovative Konzepte sind ohne Altlasten umsetzbar**
- **Weniger taktische Übergangslösung (z.B. Rückwärtsintegrationen)**

Nachteile

- **Keine Kostenersparnis realisierbar**
- **Wartung/Betrieb der Altsysteme binden länger Ressourcen**
- **Mehraufwand für gesetzlich notwendige Anpassungen**

Die Vorteile überwiegen, solange…
…die Aufwände in die Legacy Systeme minimal gehalten werden können
…die Komplexität des Gesamtsystems kontrolliert werden kann

Das bedeutet aber auch…

Wir machen wenn immer möglich keine reine Technologie Ablösung

Betrieb der Altsysteme muss längerfristig sicher gestellt sein

Das bringt Risiken, welche gemanagt werden müssen

	Ressourcen	Technologie	Anpassbarkeit	Kosten
Smalltalk	🔴	🔴	🟡	🟢
Cobol (IBM Mainframe)	🔴	🟢	🟡	🟢
Siebel	🔴	🟢	🟡	🟢

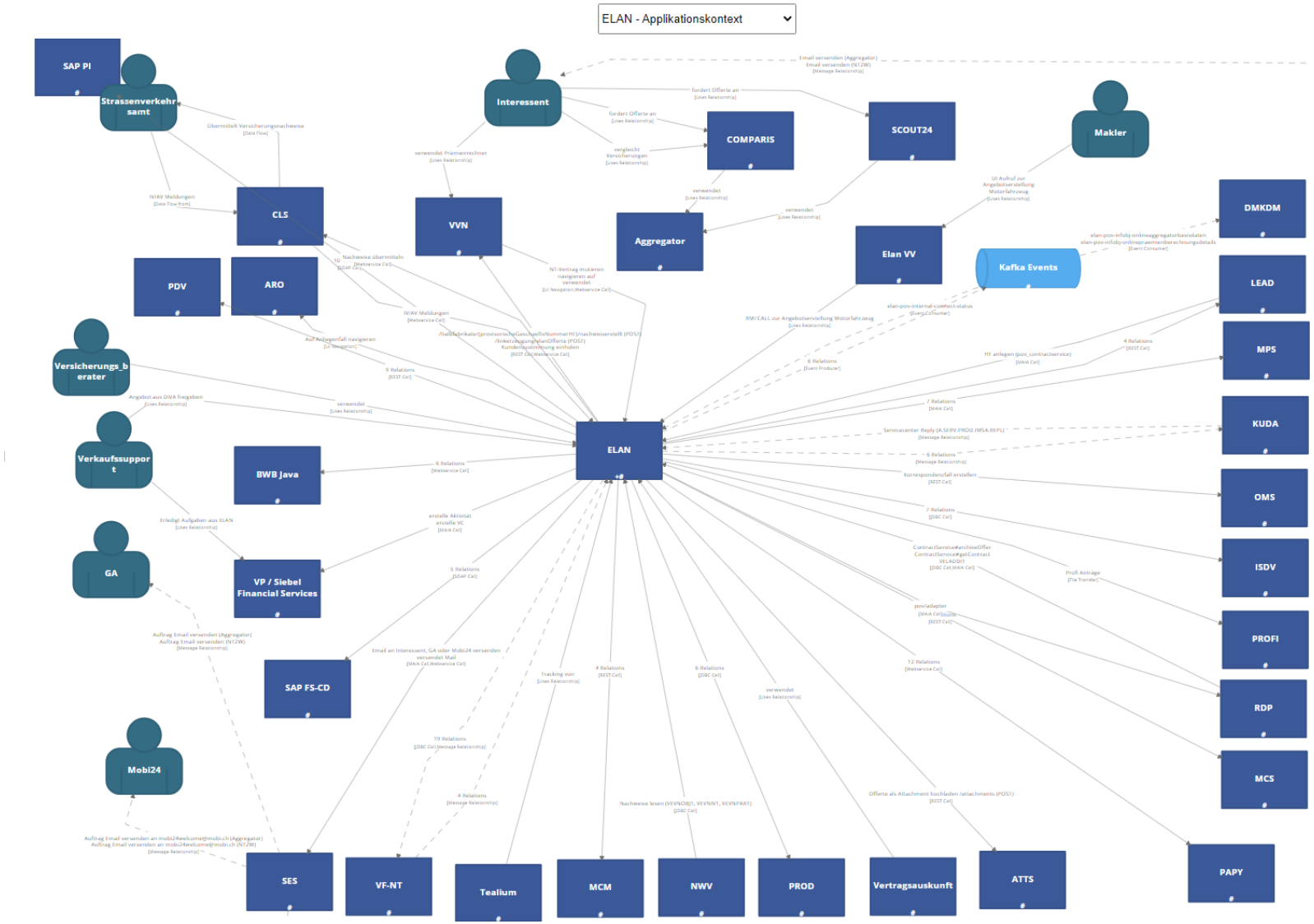
Wichtigste Massnahmen für betriebskritische Risiken

Risiken	Massnahmen
Ressourcen	• Sicherstellen dass die Schlüsselressourcen erhalten bleiben • Outsourcing von Wartung / Weiterentwicklung und Betrieb sofern dies möglich und sinnvoll ist
Technologie	• Vorbereitung einer forcierten Technologie Ablösung, welche nur im Notfall implementiert wird.

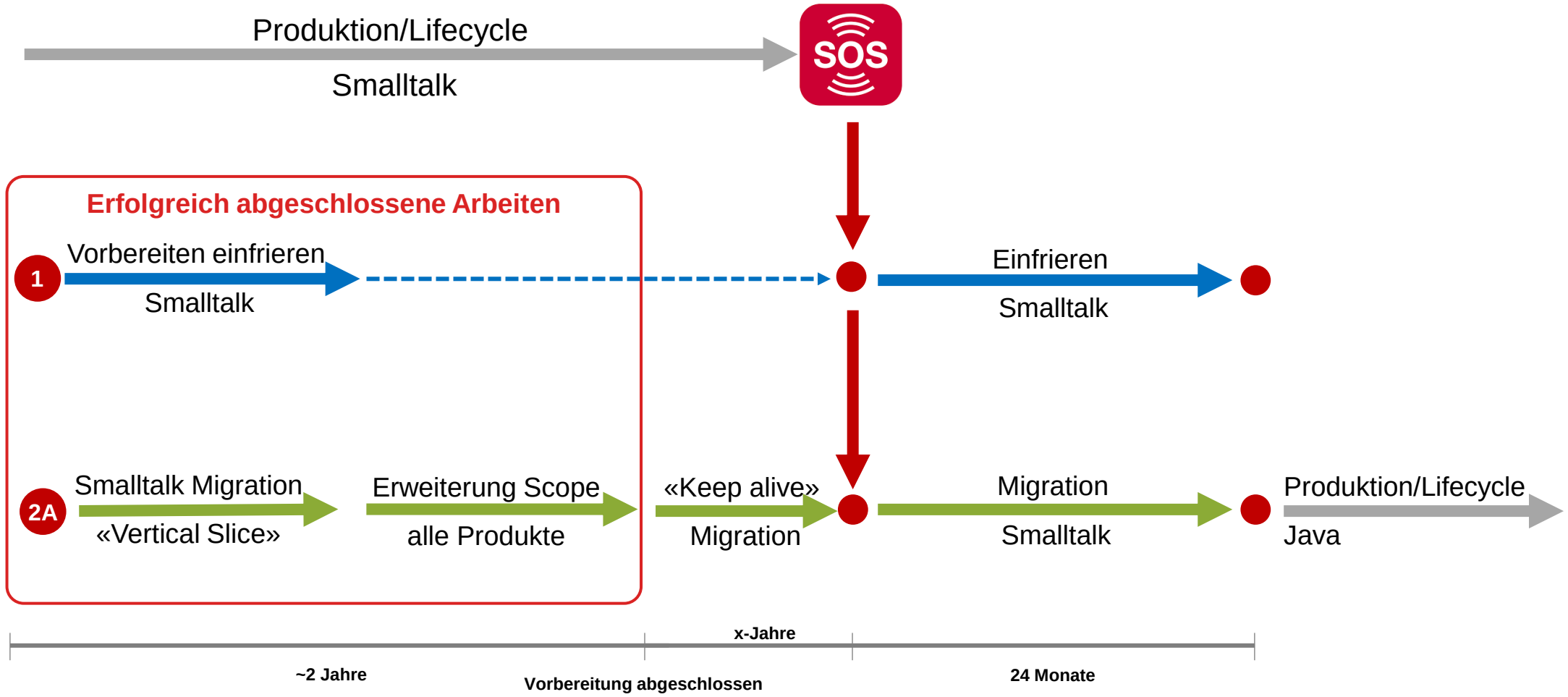
Die Smalltalk Plattform ist unser grösstes Betriebsrisiko

Massnahme:
Vorbereitung einer forcierten eins- zu eins Migration nach Java

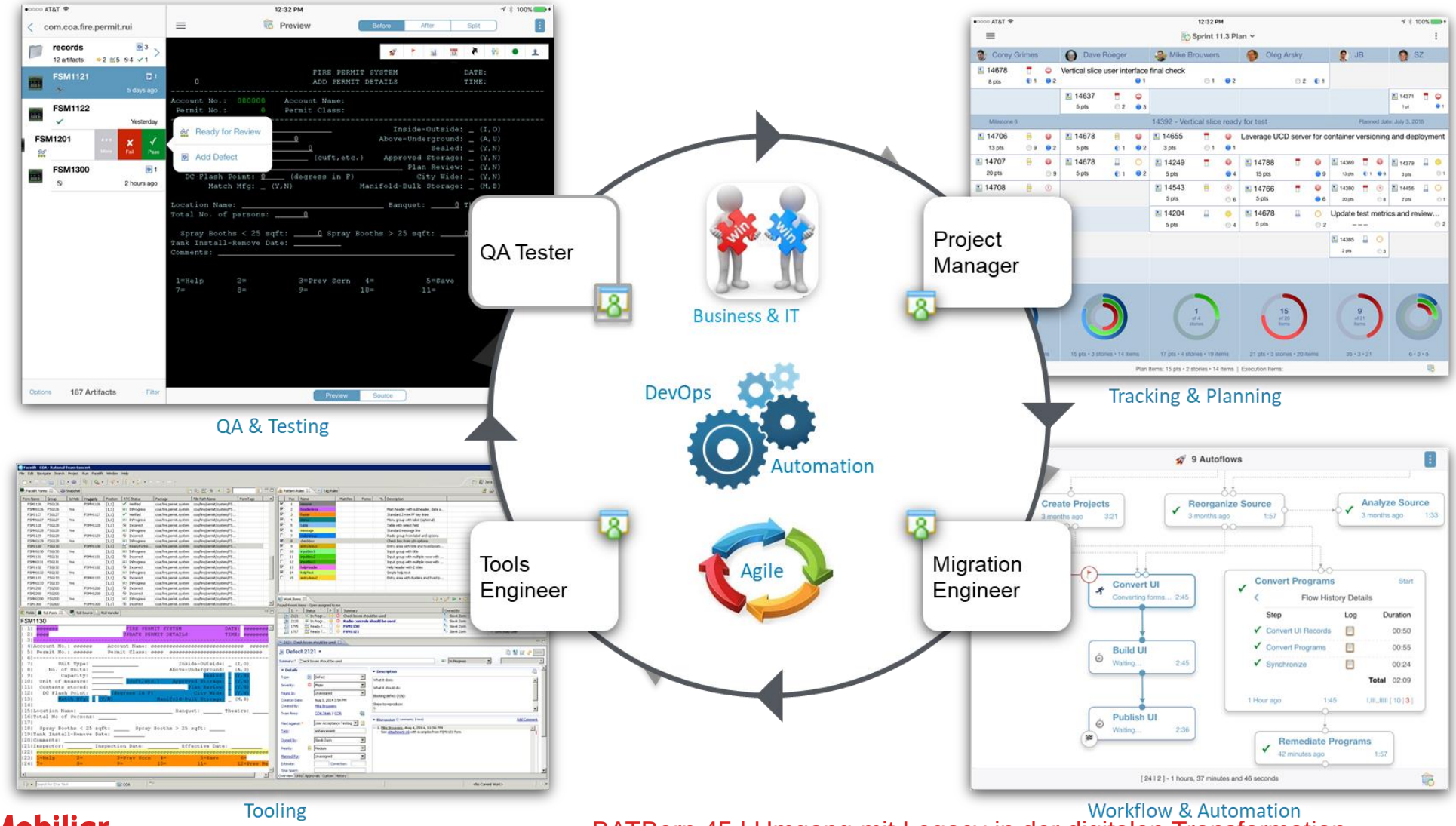
Warum nur vorbereiten und Migration nicht durchziehen?



Vorgehen der eins zu eins Migration



Teilautomatisierte Migration durch Synchrony mit ihrer Modernization Lifecycle Platform — MLP™



Der Ansatz funktioniert: Smalltalk- vs. Java-Code…

Smalltalk

```
convertTableCalculation
    "The table rules in peap are constructs connected to the specification over an SBBR containing the link to the attribute concerned.
    The table rules are SBBs with Identifiers 1718; its direct twin, 1717, defines a similar calculation, but defined using a formula expression.
    After successfully interpreting the rules, discard the SBBs (that is, disconnect them)"

    |fc|

    fc := MoProductLoader forwardChaining.

    MoSBBAllTableCalculationCodes do: [:ruleType |
        (self sbbs at: ruleType ifAbsent: [#()]) do: [:tableRules |
            tableRules do: [:tableRule | |specifications|
                specifications := tableRule getEntitiesRoleNamed: 'isSBBfor'.
                specifications do: [:specification | |aspectType|
                    aspectType := (specification getRelationNamed: 'hasSBB' to: tableRule) ifNotNil: [:relation | relation aspectAt: 'attributeType'].
                    (specification aspectAt: aspectType asString) expression
                        ifNil: [(specification aspectAt: aspectType asString) expression: MoHistorizedExpressionCollection new]
                        ifNotNil: [:expression |
                            (expression class == MoHistorizedExpressionCollection) ifFalse: [
                                Transcript cr; show: '#convertTableCalculation: Replacing default expression '
                                    , expression printString, ' with conversion table on '
                                    , (specification aspectAt: aspectType asString) printString.
                                (specification aspectAt: aspectType asString) expression: MoHistorizedExpressionCollection new]].
                    (specification aspectAt: aspectType asString) expression add: tableRule.
                    (tableRule aspectAt: 'Table') inputAttributes do: [:inputAttribute |
                        (fc at: inputAttribute asString ifAbsent: [fc at: inputAttribute asString put: Set new]) addAll: (
                            (tableRule aspectAt: 'Table') outputAttributes collect: [:outputAttribute | outputAttribute asString])]].
                tableRule disconnectAllRelations]]].
```

Java

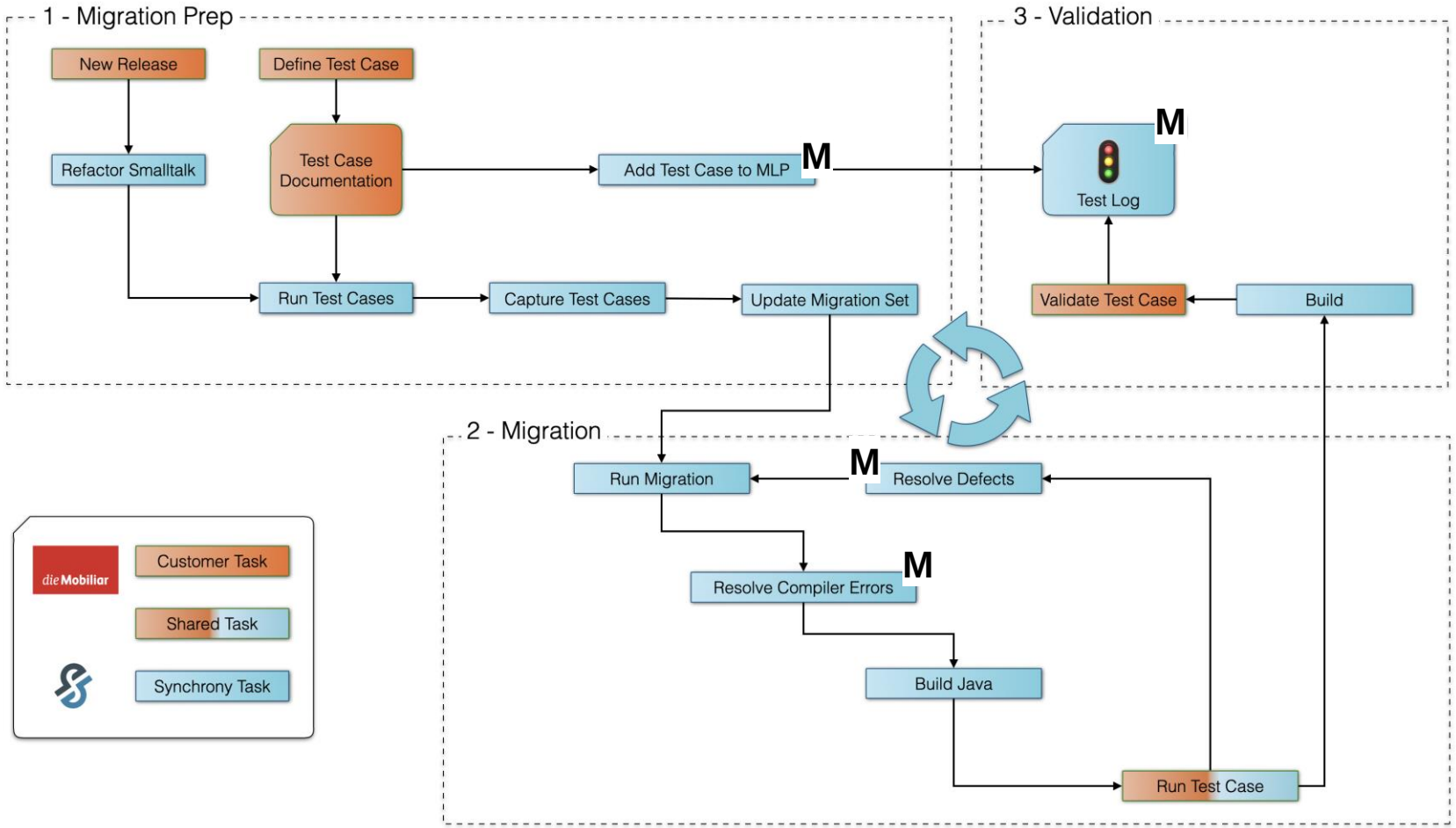
```
@Override
public MoProductLoader.Metaclass convertTableCalculation() {
    /*
     * The table rules in peap are constructs connected to the specification over an SBBR containing the link to
     * the attribute concerned. The table rules are SBBs with Identifiers 1718; its direct twin, 1717, defines a
     * similar calculation, but defined using a formula expression. After successfully interpreting the rules,
     * discard the SBBs (that is, disconnect them)
     */

    Dictionary<String, RichIterable<String>> fc = MoProductLoader.clazz.forwardChaining();

    for (Integer ruleType : Pool_MoPSConstants.clazz.instance().MoSBBAllTableCalculationCodes()) {
        for (MutableList<MoSBB> tableRules : ((RichIterable<MutableList<MoSBB>>) CollectionUtils
                .getIfAbsent(this.sbbs(), ruleType, Block.of(Lists.mutable.with()))) {
            for (MoSBB tableRule : tableRules) {
                IdentitySet<MoIAAEntity> specifications = tableRule.getEntitiesRoleNamed("isSBBfor");
                for (MoIAAEntity specification : specifications) {
                    Integer aspectType = ObjectUtils.ifNotNil(
                            specification.getRelationNamedTo("hasSBB", tableRule),
                            Block.of((MoRelation relation) -> (Integer) relation.aspectAt("attributeType")));
                    ObjectUtils.ifNilIfNotNil(
                            ((MoAspect) specification.aspectAt(ObjectUtils.asString(aspectType))).expression(),
                            Block.of(() -> {
                                ((MoAspect) specification.aspectAt(ObjectUtils.asString(aspectType)))
                                        .expression(MoHistorizedExpressionCollection.clazz.basicNew());
                            }), Block.of((Object expression) -> {
                                if (!ObjectUtils.identityEquals(ClassRegistry.getClass(expression),
                                        MoHistorizedExpressionCollection.clazz)) {
                                    System.out.append('\n');
                                    System.out.print("#convertTableCalculation: Replacing default expression "
                                            + SyncStringUtils.printString(expression)
                                            + " with conversion table on " + SyncStringUtils.printString(
                                                    specification.aspectAt(ObjectUtils.asString(aspectType))));
                                    ((MoAspect) specification.aspectAt(ObjectUtils.asString(aspectType)))
                                            .expression(MoHistorizedExpressionCollection.clazz.basicNew());
                                }
                            }));
                    CollectionUtils.add(
                            (RichIterable) ((MoAspect) specification.aspectAt(ObjectUtils.asString(aspectType)))
                                    .expression(),
                            tableRule);
                    for (Integer inputAttribute : ((MoExpPEAPTable) tableRule.aspectAt("Table"))
                            .inputAttributes()) {
                        CollectionUtils.addAll(
                                CollectionUtils.getIfAbsent(fc, ObjectUtils.asString(inputAttribute), Block.of(
                                        () -> fc.put(ObjectUtils.asString(inputAttribute), new Set<String>()))),
                                CollectionUtils.collect(
                                        ((MoExpPEAPTable) tableRule.aspectAt("Table")).outputAttributes(),
                                        (Integer outputAttribute) -> ObjectUtils.asString(outputAttribute)));
                    }
                }
                tableRule.disconnectAllRelations();
            }
        }
    }

    return this;
}
```

...aber es ist harte Arbeit und dauerte rund 2 Jahre /2100 PT Aufwand (davon 2000PT extern durch Synchrony)



- Hohe manuelle Aufwände
- 90% der Arbeit für Migration, insb. Definition der Migrationsregeln in MLP.
- Key Painpoints:
  - Viel Zeit für Testing/Validation wegen hohen Datenmengen.
  - Aufwändiges Debuggen wegen Datenhaltung auf Mainframe.

Zusammenfassung / Fragen